

SNOWFLAKE VERSION 2.0

1. Descripción general

SNOWFLAKE-V2 es un sistema de gestión académica desarrollado en Java para administrar estudiantes, profesores, administradores, materias, grupos, horarios, inscripciones, calificaciones y credenciales.

Esta segunda versión evoluciona el proyecto original hacia una estructura más cercana a una aplicación backend real, incorporando persistencia de datos, seguridad de contraseñas, arquitectura por capas y una mejor separación de responsabilidades.

2. Propósito del proyecto

El objetivo de SNOWFLAKE-V2 es simular el funcionamiento básico de un sistema académico universitario y, al mismo tiempo aplicar conceptos de desarrollo de software como:

- Programación orientada a objetos.
- Arquitectura por capas.
- Bases de datos relacionales.
- JDBC.
- Persistencia de datos.
- Integridad referencial.
- Seguridad de contraseñas.
- Transacciones.
- Diseño modular.

3. Arquitectura del sistema

SNOWFLAKE-V2 utiliza una arquitectura organizada por capas.

ConsoleUI

Se encarga de la interacción con el usuario mediante consola, mostrando menús, solicitando información y mostrando resultados.

AcademicManagement

Coordina la lógica y los procesos principales del sistema, incluyendo validaciones, generación de códigos, gestión de usuarios y operaciones académicas.

RepositorySQL

Gestiona el acceso a la base de datos, incluyendo consultas, inserciones, actualizaciones, eliminaciones y transacciones.

Modelos del dominio

Representan las principales entidades del sistema, entre ellas:

- User
- Student
- Employee
- Teacher
- Administrator
- Subject
- Group
- Schedule
- ClassSession
- GradeRecord
- Credential

Esta separación reduce el acoplamiento y facilita el mantenimiento y futuras ampliaciones.

4. Base de datos y persistencia

SNOWFLAKE-V2 utiliza SQLite conectada mediante JDBC.

Las principales tablas son:

- users
- students
- employees
- teachers
- administrators
- subjects
- academic_groups
- enrollments
- class_sessions
- grades

Las tablas se relacionan mediante claves primarias y foráneas, permitiendo representar relaciones entre usuarios, profesores, estudiantes, materias, grupos, horarios, inscripciones y calificaciones.

5. Integridad referencial

La base de datos utiliza claves foráneas y reglas como:

ON DELETE CASCADE

Elimina automáticamente registros dependientes cuando se elimina su registro principal.

ON DELETE SET NULL

Permite conservar un registro cuando la entidad relacionada es eliminada.

Por ejemplo, un grupo puede seguir existiendo aunque su profesor sea eliminado.

6. Gestión de usuarios y roles

SNOWFLAKE-V2 utiliza tres roles principales.

Student

Puede consultar:

- Horario.
- Calificaciones.
- Credencial.

Teacher

Puede consultar:

- Horario.
- Grupos asignados.
- Estudiantes inscritos.
- Información relacionada con calificaciones.

Administrator

Puede:

- Registrar estudiantes, profesores y administradores.
- Crear materias.
- Crear grupos académicos.
- Crear sesiones de clase.
- Inscribir estudiantes.
- Actualizar información.
- Cambiar contraseñas.
- Eliminar usuarios.

7. Seguridad

Las contraseñas no se almacenan en texto plano.

SNOWFLAKE-V2 utiliza **BCrypt** para generar hashes seguros y verificar las contraseñas durante el inicio de sesión.

También utiliza `SecureRandom` para generar automáticamente contraseñas iniciales con:

- Letras mayúsculas.
- Letras minúsculas.
- Números.
- Símbolos.

Además, el sistema valida el tamaño de las contraseñas antes de procesarlas con BCrypt.

8. Gestión académica

Materias

Cada materia contiene un código único y un nombre.

Grupos académicos

Cada grupo está relacionado con una materia y puede tener un profesor asignado.

Sesiones de clase

Cada sesión contiene:

- Código de clase.
- Grupo.
- Materia.
- Día.
- Hora.

Las sesiones utilizan una clave primaria compuesta:

```
class_code + day
```

Esto permite que una misma clase utilice el mismo código en diferentes días.

Inscripciones

Los estudiantes pueden inscribirse en diferentes grupos.

La tabla `enrollments` utiliza:

```
student_code + group_code
```

como clave compuesta para evitar inscripciones duplicadas.

Calificaciones

Cada inscripción tiene asociado un registro de calificaciones con:

- First Partial.
- Second Partial.
- Third Partial.
- Final Grade.

9. Transacciones

SNOWFLAKE-V2 utiliza transacciones en operaciones que requieren múltiples cambios relacionados.

Algunos ejemplos son:

- Registro de usuarios.
- Creación de múltiples sesiones de clase.
- Inscripción de estudiantes junto con su registro de calificaciones.

Se utilizan:

- `setAutoCommit(false)`
- `commit()`
- `rollback()`

Esto evita que una operación incompleta quede almacenada parcialmente.

10. JDBC y acceso a datos

El sistema utiliza JDBC para conectar Java con SQLite.

Las principales clases utilizadas son:

Connection

Mantiene la conexión con la base de datos.

Statement

Se utiliza principalmente para crear tablas.

PreparedStatement

Permite ejecutar consultas parametrizadas de manera más segura y organizada.

ResultSet

Permite recuperar resultados de consultas y reconstruir objetos Java.

11. Convenciones del código

El proyecto sigue las convenciones habituales de Java:

- `camelCase` para variables y métodos.
- `PascalCase` para clases.

También se busca mantener separadas la interfaz, la lógica del sistema, los modelos y el acceso a datos.

12. Tecnologías utilizadas

- Java.
- SQLite.
- SQL.
- JDBC.
- BCrypt.
- SecureRandom.
- Programación orientada a objetos.
- Arquitectura por capas.
- Bases de datos relacionales.
- Transacciones.
- Integridad referencial.

13. Mejoras respecto a SNOWFLAKE-V1

SNOWFLAKE-V2 representa una evolución importante respecto a la primera versión.

Persistencia de datos

V1: la información dependía principalmente de objetos en memoria.

V2: utiliza SQLite para almacenar información de forma persistente.

JDBC

V2 incorpora JDBC para comunicar Java con la base de datos.

RepositorySQL

V2 añade una capa dedicada al acceso a datos, evitando que la lógica principal dependa directamente de SQL.

Seguridad

V2 incorpora:

- BCrypt.
- Hashing de contraseñas.
- SecureRandom.
- Generación automática de contraseñas.
- Validación del tamaño soportado por BCrypt.

Administrator

V2 incorpora un nuevo rol administrativo encargado de gestionar usuarios y elementos académicos.

Integridad referencial

Se implementan claves foráneas y reglas como:

- ON DELETE CASCADE
- ON DELETE SET NULL

Transacciones

V2 utiliza transacciones para proteger operaciones que modifican múltiples registros.

Horarios y calificaciones persistentes

La información académica ahora se almacena y recupera directamente desde la base de datos.

Arquitectura más modular

V2 presenta una separación más clara entre:

- Interfaz.
- Lógica.
- Persistencia.
- Modelos.

14. Posibles mejoras futuras

Entre las principales mejoras futuras se encuentran:

- Crear una interfaz gráfica o aplicación web.
- Implementar una API REST.
- Migrar a PostgreSQL o MySQL.
- Añadir pruebas automatizadas.
- Implementar logging.
- Añadir migraciones de base de datos.
- Mejorar permisos y recuperación de contraseña.
- Mejorar el modelo de horarios.
- Incorporar nuevas entidades académicas.

ENGLISH VERSION

SNOWFLAKE-V2

1. General Overview

SNOWFLAKE-V2 is an academic management system developed in Java to manage students, teachers, administrators, subjects, academic groups, schedules, enrollments, grades, and credentials.

This second version evolves the original project toward a structure closer to a real backend application by introducing data persistence, password security, layered architecture, and clearer separation of responsibilities.

2. Project Purpose

SNOWFLAKE-V2 aims to simulate the basic operation of a university academic system while applying concepts such as:

- Object-oriented programming.
- Layered architecture.
- Relational databases.
- JDBC.
- Data persistence.
- Referential integrity.
- Password security.
- Transactions.
- Modular design.

3. System Architecture

SNOWFLAKE-V2 uses a layered architecture.

ConsoleUI

Handles user interaction through the console.

AcademicManagement

Coordinates system logic, validations, code generation, user management, and academic operations.

RepositorySQL

Handles database access, including queries, insertions, updates, deletions, and transactions.

Domain Models

The main entities include:

- User
- Student
- Employee
- Teacher
- Administrator
- Subject
- Group
- Schedule
- ClassSession
- GradeRecord

- Credential

This separation reduces coupling and improves maintainability.

4. Database and Persistence

SNOWFLAKE-V2 uses SQLite connected through JDBC.

The main tables are:

- users
- students
- employees
- teachers
- administrators
- subjects
- academic_groups
- enrollments
- class_sessions
- grades

Primary and foreign keys are used to represent relationships between academic entities.

5. Referential Integrity

The database uses foreign keys and deletion rules such as:

ON DELETE CASCADE

Automatically removes dependent records.

ON DELETE SET NULL

Keeps a record while removing its reference to a deleted entity.

6. User Roles

Student

Can view:

- Schedule.
- Grades.
- Credential.

Teacher

Can view:

- Schedule.
- Assigned groups.
- Enrolled students.
- Grade-related information.

Administrator

Can:

- Register users.
- Create subjects and groups.
- Create class sessions.
- Enroll students.
- Update information.
- Change passwords.
- Delete users.

7. Security

Passwords are never stored in plain text.

SNOWFLAKE-V2 uses **BCrypt** for password hashing and verification.

`SecureRandom` is used to generate initial passwords containing:

- Uppercase letters.
- Lowercase letters.
- Numbers.
- Symbols.

The system also validates password size before processing it with BCrypt.

8. Academic Management

Subjects

Each subject contains a unique code and name.

Academic Groups

Each group belongs to a subject and may have an assigned teacher.

Class Sessions

Each session contains:

- Class code.
- Group.
- Subject.
- Day.
- Time.

A composite key is used:

`class_code + day`

Enrollments

Student-group relationships are stored using:

`student_code + group_code`

Grades

Each enrollment has a grade record containing:

- First Partial.
- Second Partial.
- Third Partial.
- Final Grade.

9. Transactions

Transactions are used for operations that require multiple related database changes.

Examples include:

- User registration.
- Creation of multiple class sessions.
- Student enrollment and grade record creation.

The system uses:

- `setAutoCommit(false)`
- `commit()`
- `rollback()`

10. JDBC and Data Access

The main JDBC components used are:

- `Connection`
- `Statement`
- `PreparedStatement`
- `ResultSet`

These components allow Java to interact with SQLite and reconstruct application objects from stored data.

11. Code Conventions

The project follows standard Java naming conventions:

- `camelCase` for variables and methods.
- `PascalCase` for classes.

Responsibilities are separated between interface, logic, persistence, and models.

12. Technologies Used

- Java.
- SQLite.
- SQL.
- JDBC.
- BCrypt.
- SecureRandom.
- Object-oriented programming.
- Layered architecture.
- Relational databases.
- Transactions.
- Referential integrity.

13. Improvements over SNOWFLAKE-V1

Data Persistence

V1: data mainly existed in memory.

V2: SQLite provides persistent storage.

JDBC

V2 introduces JDBC for Java-database communication.

RepositorySQL

V2 introduces a dedicated repository layer.

Security

V2 introduces:

- BCrypt.
- Password hashing.
- SecureRandom.
- Automatic password generation.
- BCrypt size validation.

Administrator Role

V2 introduces an administrator role for user and academic management.

Referential Integrity

V2 uses foreign keys and:

- ON DELETE CASCADE
- ON DELETE SET NULL

Transactions

Transactions protect operations involving multiple database changes.

Persistent Academic Information

Schedules, enrollments, and grades are now stored directly in the database.

Improved Architecture

V2 introduces clearer separation between:

- Interface.
- Logic.
- Persistence.
- Models.

14. Future Improvements

Possible future improvements include:

- Graphical or web interface.
- REST API.
- PostgreSQL or MySQL migration.
- Automated testing.
- Logging.
- Database migrations.
- Improved permissions.
- Password recovery.
- Improved schedule modeling.
- Additional academic entities.